

TASK -2 (Moving Booking System)

Assumptions and Brief Guide to interact with System:

System Overview

The Movie Booking System has two types of users:

1. Manager: Manages movies in the system.
2. Customer: Books tickets, views movies, and manages their bookings.

The system is password-protected for both Managers and Customers to ensure secure operations.

Step 1: Choose User Role

When the system starts, you are prompted to select whether you are a Manager or a Customer.

- Enter 1 for Manager.
- Enter 2 for Customers.

Manager Guide

1. Login as Manager:
 - Enter the fixed manager password: "IamBoss".(case-sensitive)
 - If the password is incorrect, the system will display an error and allow you to try again.
 - After successful login, you can access the following features.
2. Manager Functions:
 - Add a Movie:
 - Enter the movie details:
 - Movie ID (which should be an integer unique for each movie)
 - Title (Name of the Movie)
 - Start Time (HH:MM) (24 Hour format)
 - Duration (in minutes) (eg. 120,148)
 - Total number of Seats
 - Ticket Price (which can be any real value i.e. decimal values also)
 - The system checks for:
 - Duplicate movie IDs.
 - Duplicate movie titles.
 - Overlapping movie times (with a 15-minute buffer) to avoid schedule conflicts.
 - If any issues are found, the manager is notified. Otherwise, the movie is added successfully.
 - Remove a Movie:
 - Enter the Movie ID of the movie to be removed.
 - The system will confirm if the movie is removed or display an error if the ID is invalid.
 - View All Movies:

- Displays a list of all movies with details like ID, title, time, ticket price, and available seats.
 - View All Customer Bookings:
 - Displays a detailed list of all bookings made by customers for every movie.
 - Exit Manager Portal: Exits the manager's session and returns to the main menu.
-

Customer Guide

1. Login as Customer:

- Your **Customer ID** is a unique integer between **1 and 10000**. This ID will serve as your unique identifier in the system. (since I have assumed the upper limit for the number of customers who can login to the system as 10000 and the array index is used as customer's ID).
- **Set Password (First Login Only):**
On your **first login**, you will be prompted to set a Password. Choose a strong password. You will need to confirm your password after setting it to complete the registration process.
- **Subsequent Logins:**
After setting your password on the first login, you will use the same password for all future logins. Enter your Customer ID and Password to log in.

2. Customer Functions:

- **View Available Movies:**
 - Displays a list of all movies with details like ID, title, time, ticket price, and available seats.
 - If no movies are available, the system will notify the user.
- **Book a Ticket:**
 - Enter the Movie ID of the movie you wish to book.
 - If there are available seats and sufficient balance, the ticket is booked, and your account is charged.
 - If booking fails (e.g., insufficient balance or no seats available), an error is displayed.
- **Cancel a Ticket:**
 - Enter the Movie ID for which you want to cancel your booking.
 - If you have a booking for that movie, the system refunds your account and cancels the ticket. If no booking is found, the system notifies the user.
- **View Your Bookings:**
 - Displays a list of movies you have booked tickets for, including details like Movie ID and title.
- **View Account Balance:**
 - Displays your current account balance.

- I have assigned each customer randomly a real-valued account balance between Rs 5000 and 10000. The balance is used for ticket bookings.
 - **Exit Customer Portal:** Exits the customer's session and returns to the main menu.
-

System Features

- **Password Security:**

Both Manager and Customer have their own password-protected systems for security.

The Manager's password is fixed, while customers can set their password during the first login.

- **Error Handling:**

- Invalid inputs (e.g., wrong Movie ID, incorrect password) prompt error messages and allow retries.

- **Exit System:**

- At any point, users can exit by choosing the option to log out or terminate the program.
-

2. Code (in C++):

```
#include<bits/stdc++.h>
using namespace std;

const int MAX_MOVIES = 50; // Upper bound on number of movies
const int MAX_CUSTOMERS = 10000; // Upper bound on number of customers

void pressEnterToContinue() {
    cout << "\nPress Enter to continue...";
    cin.ignore(); // Ignore the previous newline character from any
prior input
    cin.get();    // Wait for the user to press Enter
}

class Movie {
private:
    int ID;
    string title;
    int duration; // in minutes
```

```

    string start_time;
    int capacity;
    int booked;
    double ticket_price;
    vector<int> customers_booked;

public:
    Movie() : ID(0), title(""), duration(0), start_time(""),
capacity(0), booked(0), ticket_price(0.0) {}

    Movie(int id, const string& t, int d, const string& st, int cap,
double price)
        : ID(id), title(t), duration(d), start_time(st), capacity(cap),
booked(0), ticket_price(price) {}

    string getStartTime() const { return start_time; }
    int getDuration() const { return duration; }

    void inputMovieDetails() {
        cout << "\n===== Add New Movie =====\n";
        cout << "Please enter the following movie details:\n";

        cout << "Movie ID (unique identifier) (integer): ";
        cin >> ID;
        cin.ignore();

        cout << "Movie Title: ";
        getline(cin, title);

        cout << "Movie Duration (in minutes, e.g., 120): ";
        cin >> duration;
        cin.ignore();

        cout << "Movie Start Time (HH:MM format, e.g., 19:30): ";
        getline(cin, start_time);

        cout << "Total Seats Available: ";
        cin >> capacity;

        cout << "Ticket Price (Rs): ";
        cin >> ticket_price;
    }

```

```

        cout << "\n[+] Movie \"" << title << "\"" added
successfully!\n";
        pressEnterToContinue();
    }

    bool bookTicket(int customer_id, double& balance) {
        if (booked >= capacity) {
            cout << "[!] Sorry! No seats available for \"" << title <<
"\n".\n";
            pressEnterToContinue();
            return false;
        }
        if (balance < ticket_price) {
            cout << "[!] Insufficient funds to book the ticket for \""
<< title << "\n".\n";
            pressEnterToContinue();
            return false;
        }

        balance -= ticket_price;
        customers_booked.push_back(customer_id);
        booked++;
        cout << "[+] Ticket booked successfully for \"" << title <<
"\n!\n";
        pressEnterToContinue();
        return true;
    }

    void cancelTicket(int customer_id, double& balance) {
        auto it = find(customers_booked.begin(),
customers_booked.end(), customer_id);
        if (it != customers_booked.end()) {
            customers_booked.erase(it);
            booked--;
            balance += ticket_price;
            cout << "[+] Ticket cancelled successfully for \"" << title
<< "\n".\n";
        } else {
            cout << "[!] No booking found for this movie.\n";
        }
        pressEnterToContinue();
    }
}

```

```

void display() const {
    cout << "\n===== Movie Details =====\n";
    cout << "Movie ID      : " << ID << endl;
    cout << "Title          : " << title << endl;
    cout << "Start Time     : " << start_time << endl;
    cout << "Duration       : " << duration << " minutes\n";
    cout << "Available Seats: " << (capacity - booked) << "/" <<
capacity << endl;
    cout << "Ticket Price : Rs " << fixed << setprecision(2) <<
ticket_price << endl;
    cout << "===== \n";
    pressEnterToContinue();
}

// Getter methods
int getID() const { return ID; }
string getTitle() const { return title; }
const vector<int>& getCustomersBooked() const { return
customers_booked; }
};

class Manager {
private:
    string password;
    double account_balance;

    // Helper function to convert time string to minutes
    int timeToMinutes(const string& time) {
        int hours = stoi(time.substr(0, 2));
        int minutes = stoi(time.substr(3, 2));
        return hours * 60 + minutes;
    }

    // Helper function to check if two movie times overlap
    bool isTimeOverlap(const Movie& existing, const Movie& newMovie) {
        // Convert start times to minutes
        int existing_start = timeToMinutes(existing.getStartTime());
        int new_start = timeToMinutes(newMovie.getStartTime());

        // Calculate end times by adding movie durations
        int existing_end = existing_start + existing.getDuration();
        int new_end = new_start + newMovie.getDuration();
    }
}

```

```

        // Detailed overlap check with buffer time (15 minutes)
        const int BUFFER_TIME = 15; // 15 minutes buffer between
movies

        // Check for complete overlap scenarios
        bool overlap = (
            (new_start < existing_end + BUFFER_TIME) && // New movie
starts too early
            (new_end > existing_start - BUFFER_TIME) // New movie
ends too late
        );

        return overlap;
    }

public:
    // Constructor remains the same as in the original code
    Manager(string pass) : password(pass), account_balance(0) {}

    // Validate password method
    bool validatePassword(string input_password) {
        return input_password == password;
    }

    void addMovie(vector<Movie>& movies) {
        if (movies.size() >= MAX_MOVIES) {
            cout << "[!] Maximum movie limit (" << MAX_MOVIES << ")
reached. Cannot add more movies.\n";
            pressEnterToContinue();
            return;
        }

        Movie new_movie;
        new_movie.inputMovieDetails();

        // Check for time slot conflict and duplicate IDs
        for (const auto& movie : movies) {
            if (movie.getTitle().empty()) continue;

            // Check for duplicate ID
            if (new_movie.getID() == movie.getID()) {
                cout << "[!] A movie with this ID already exists!\n";
                pressEnterToContinue();
            }
        }
    }

```

```

        return;
    }

    // Check for duplicate title
    if (new_movie.getTitle() == movie.getTitle()) {
        cout << "[!] A movie with this title already exists!\n";
        pressEnterToContinue();
        return;
    }

    // New check: Prevent same start time for different movies
    if (new_movie.getStartTime() == movie.getStartTime()) {
        cout << "[!] A movie with the same start time already
exists!\n";

        cout << "Existing Movie: " << movie.getTitle()
            << " starts at " << movie.getStartTime() << endl;
        cout << "New Movie: " << new_movie.getTitle()
            << " starts at " << new_movie.getStartTime() << endl;
        cout << "[!] New movie was not added to the system.\n";
        pressEnterToContinue();
        return;
    }

    // Check for time slot conflict with 15-minute buffer
    if (isTimeOverlap(movie, new_movie)) {
        cout << "[!] Time slot conflict with existing movie \""
            << movie.getTitle() << "\". Cannot add movie.\n";
        cout << "Conflict details:\n";
        cout << "Existing Movie: " << movie.getTitle()
            << " (Start: " << movie.getStartTime()
            << ", Duration: " << movie.getDuration() << "
mins)\n";

        cout << "New Movie: " << new_movie.getTitle()
            << " (Start: " << new_movie.getStartTime()
            << ", Duration: " << new_movie.getDuration() << "
mins)\n";

        cout << "[!] New movie was not added to the system.\n";
        pressEnterToContinue();
        return;
    }
}

}

movies.push_back(new_movie);

```

```

}

void removeMovie(vector<Movie>& movies) {
    if (movies.empty()) {
        cout << "[!] No movies available to remove.\n";
        pressEnterToContinue();
        return;
    }

    int movie_id;
    cout << "Enter Movie ID to remove: ";
    cin >> movie_id;

    auto it = find_if(movies.begin(), movies.end(),
        [movie_id](const Movie& movie) { return movie.getID() ==
movie_id; });

    if (it != movies.end()) {
        cout << "[*] Removing movie: \"" << it->getTitle() <<
"\n\n";

        movies.erase(it);
        cout << "[+] Movie removed successfully!\n";
    } else {
        cout << "[!] Movie not found!\n";
    }
    pressEnterToContinue();
}

void viewBookings(const vector<Movie>& movies) {
    cout << "\n===== Current Movie Bookings =====\n";
    bool has_bookings = false;

    for (const auto& movie : movies) {
        auto customers = movie.getCustomersBooked();
        if (!customers.empty()) {
            has_bookings = true;
            cout << "Movie: " << movie.getTitle() << endl;
            cout << "Booked by Customers: ";
            for (int customer_id : customers) {
                cout << customer_id << " ";
            }
            cout << "\n\n";
        }
    }
}

```

```

    }

    if (!has_bookings) {
        cout << "[*] No bookings found for any movies.\n";
    }

    cout << "=====\n";
    pressEnterToContinue();
}

void showManagerMenu() {
    cout << "\n===== Manager Dashboard =====\n";
    cout << "1. Add New Movie\n";
    cout << "2. Remove Movie\n";
    cout << "3. View All Bookings\n";
    cout << "4. Exit Manager Portal\n";
    cout << "=====\n";
}
};

class Customer {
private:
    int ID;
    string password;
    double account_balance;
    bool is_first_login;

public:
    Customer(int id, const string& pass, double balance)
        : ID(id), password(pass), account_balance(balance),
is_first_login(true) {}

    bool validatePassword(const string& input_password) {
        return input_password == password;
    }

    void setPassword() {
        if (is_first_login) {
            cout << "\nWelcome, Customer ID " << ID << "! It seems this
is your first login.";
            cout << "\nPlease set your password: ";
            cin >> password;
            is_first_login = false;
            cout << "[+] Password set successfully!\n";

```

```

    }
}

void viewMovies(const vector<Movie>& movies) {
    if (movies.empty()) {
        cout << "[!] No movies currently available.\n";
        pressEnterToContinue();
        return;
    }

    cout << "\n===== Available Movies =====\n";
    for (const auto& movie : movies) {
        movie.display();
    }
}

void bookTicket(vector<Movie>& movies) {
    cout << "\n===== Book a Movie Ticket =====\n";
    cout << "Your Current Balance: Rs" << account_balance << endl;

    int movie_id;
    cout << "Enter Movie ID to book: ";
    cin >> movie_id;

    for (auto& movie : movies) {
        if (movie.getID() == movie_id) {
            movie.bookTicket(ID, account_balance);
            return;
        }
    }

    cout << "[!] Movie ID not found!\n";
    pressEnterToContinue();
}

void cancelTicket(vector<Movie>& movies) {
    cout << "\n===== Cancel a Movie Ticket =====\n";

    int movie_id;
    cout << "Enter Movie ID to cancel ticket: ";
    cin >> movie_id;

    for (auto& movie : movies) {
        if (movie.getID() == movie_id) {
            movie.cancelTicket(ID, account_balance);
        }
    }
}

```

```

        return;
    }
}

cout << "[!] Movie ID not found!\n";
pressEnterToContinue();
}

void viewBookings(const vector<Movie>& movies) {
    cout << "\n===== Your Movie Bookings =====\n";
    bool has_booking = false;

    for (const auto& movie : movies) {
        auto customers = movie.getCustomersBooked();
        if (find(customers.begin(), customers.end(), ID) !=
customers.end()) {
            has_booking = true;
            cout << "Movie: " << movie.getTitle() << "\n";
        }
    }

    if (!has_booking) {
        cout << "[*] No bookings found.\n";
    }
    pressEnterToContinue();
}

void showCustomerMenu() {
    cout << "\n===== Customer Dashboard =====\n";
    cout << "1. View Movies\n";
    cout << "2. Book Ticket\n";
    cout << "3. Cancel Ticket\n";
    cout << "4. View Your Bookings\n";
    cout << "5. Exit Customer Portal\n";
    cout << "===== \n";
}

// Getter methods
int getID() const { return ID; }
double getBalance() const { return account_balance; }
};

int main() {
    vector<Movie> movies;

```

```

Manager manager("IamBoss"); // Manager's password
vector<Customer> customers;

// Create max customers with random balances
random_device rd;
mt19937 gen(rd());
uniform_real_distribution<> dis(5000.0, 10000.0);

for (int i = 1; i <= MAX_CUSTOMERS; ++i) {
    customers.push_back(Customer(i, "", dis(gen))); // Initially,
no password is set
}

while (true) {
    cout << "\n===== Welcome to the Movie Booking System! =====\n";
    cout << "1. Manager Login\n";
    cout << "2. Customer Login\n";
    cout << "3. Exit System\n";
    cout << "=====\n";
    cout << "Enter your choice: ";
    int choice;
    cin >> choice;

    if (choice == 1) {
        // Manager login logic
        string manager_password;
        cout << "\nEnter Manager Password: ";
        cin >> manager_password;

        if (manager.validatePassword(manager_password)) {
            cout << "\nGreetings Manager! Login successful.\n";
while (true) {
                manager.showManagerMenu();
                int manager_choice;
                cin >> manager_choice;

                if (manager_choice == 1) {
                    manager.addMovie(movies);
                } else if (manager_choice == 2) {
                    manager.removeMovie(movies);
                } else if (manager_choice == 3) {
                    manager.viewBookings(movies);
                } else if (manager_choice == 4) {

```

```

        cout<< "GoodBye Manager! Logout Successful";
        break;
    } else {
        cout << "[!] Invalid choice!\n";
        pressEnterToContinue();
    }
}
} else {
    cout << "[!] Incorrect Manager Password!\n";
    pressEnterToContinue();
}
} else if (choice == 2) {
    // Customer login logic
    int customer_id;
    string customer_password;

    cout << "\nEnter Customer ID: ";
    cin >> customer_id;

    if (customer_id > 0 && customer_id <= MAX_CUSTOMERS) {
        Customer& customer = customers[customer_id - 1];

        customer.setPassword();

        cout << "\nEnter Password: ";
        cin >> customer_password;

        if (customer.validatePassword(customer_password)) {
            cout << "\nWelcome, Customer ID " << customer_id <<
"! Login successful.\n";
            while (true) {
                customer.showCustomerMenu();
                int customer_choice;
                cin >> customer_choice;

                if (customer_choice == 1) {
                    customer.viewMovies(movies);
                } else if (customer_choice == 2) {
                    customer.bookTicket(movies);
                } else if (customer_choice == 3) {
                    customer.cancelTicket(movies);
                } else if (customer_choice == 4) {
                    customer.viewBookings(movies);

```

```
        } else if (customer_choice == 5) {
            break;
        } else {
            cout << "[!] Invalid choice!\n";
            pressEnterToContinue();
        }
    }
} else {
    cout << "[!] Incorrect Password!\n";
    pressEnterToContinue();
}
} else {
    cout << "[!] Invalid Customer ID!\n";
    pressEnterToContinue();
}
} else if (choice == 3) {
    cout << "Exiting the system. Goodbye!\n";
    break;
} else {
    cout << "[!] Invalid choice!\n";
    pressEnterToContinue();
}
}

return 0;
}
```